IOT452U Assessment Brief – Individual Coursework

Module Code	**IOT452U**
Module Title	**Software Engineering Tools and Techniques and Practice**
Module Organiser	**Alim Ul Gias**
Assessment	**Individual Coursework**
Weighting (%)	**50%**
Deadline	**Tuesday, 19th May 2026, by 11:59pm**

Project Context and Scenario

ADigital IDprogramme has been initiated to support the management and use of digital identities across a federated ecosystem of organisations. The programme requires a backend system that allows a central authority to manage digital identities, while enabling other authorised organisations to make controlled use of identity information.

Within this ecosystem, a central authority (for example, a home or interior ministry) is solely responsible for the creation, update, and status management of Digital IDs. Other organisations, such as tax services, driving licence authorities, welfare services, immigration-related bodies, local authorities, banks, or employers, do not modify identity data. Instead, they interact with the system to verify identities, check identity status, or obtain limited identity information relevant to their needs.

Participating organisations access the Digital ID platform through dedicated portals that are developed and provided by the central authority, with each portal aligned to the role and needs of the organisation using it. Requests from these portals are processed by a shared Digital ID platform. Organisations do not communicate directly with one another.

At the coreofthesystemisDigitalIDlifecyclemanagement.Thesystemmustsupport the creationofnewidentities,updatestopermittedidentityattributes,andchangesto identity status in accordance with defined rules. Certain attributes are immutable, while othersmaybemodifiedonlybythecentralauthority.EachDigitalIDhasan associated status indicating whether it is currently valid for use, and this status directly affects howtheidentitymaybeusedbyconsumingorganisations.Operationsmustbe applied deterministicallysothateachDigitalIDremainsinavalidandconsistentstate, including when operations are repeated or conflict with the current status, for example when attemptingtoupdatearevokedDigitalID.

For organisationsthatconsumeidentityinformation,thesystemsupportsarangeof identity verification and lookup scenarios. While all organisations interact with the same underlying platform, the information requested and the form of the response may differ depending on organisational needs. Some organisations may require confirmation that a DigitalIDexistsandiscurrentlyvalid,whileothersmayneedtoevaluateadditional conditions relevant to their domain.

For example,ataxauthoritymaycheckthataDigitalIDexists,isactive,andhasnot been suspendedduringagivenreportingperiodbeforeprocessingareturn.Adriving licence authority may verify that a Digital ID is active and that any additional eligibility conditions defined for licensing purposes are satisfied before issuing or renewing a licence, for example that the identity is not subject to a temporary restriction. In contrast, an employer or bank may require only a limited verification response indicating whether a Digital ID is currently valid at the time of the request, without access to any additional identity attributes or historical information.

The systemappliesvalidation,organisation-levelauthorisation,andbusinessrulesto all incomingrequestsbeforeperforminganyoperation.Requeststhatattempt unauthorised actions, violate identity rules, or conflict with the current state of a Digital ID are rejectedinadefinedandconsistentmanner.Thisbehaviourmustbe independent of the number of participating organisations or portals. Key actions such as identitycreation,updates,statuschanges,andverificationrequestsarerecordedto allow systembehaviourtobeexamined.

Identity management, including creation, update, and status changes, and identity consumption, including verification and lookup, are treated as distinct system capabilities and must be handled separately by the system.

The projectisintendedtobeimplementedasaconsole-basedbackendsystem.There is no requirementforauserinterfaceorweblayer.Thefocusisonsystembehaviour and structureratherthanonuserinterfaceconcernsortheuseoflargeapplication frameworks.

Queen Mary University of London

1. Assessment Overview

Thiscoursework is an individual final assessment for the module. It is intended to reflect the overall learning achieved across the module and to evaluate how learners apply core software engineering practices independently when designing and implementing a backend system.

You will design and develop a software system based on the scenario described above. The emphasis is not on building a large or feature-rich product, but on demonstrating correct system behaviour, clear structure, and sound development practices.

Throughout this assessment, you are expected to apply concepts developed during the module, including software architecture, code organisation, automated testing, version control, and clear technical communication. As no separate technical report is required, the code itself should clearly express system structure, behaviour, and design decisions through readable and well-organised implementation.

The system does not need to be large or complex. A smaller system is acceptable provided it demonstrates sufficient behaviour, is clearly structured, and shows evidence of organised development.

You must implement your system in either Python or Java.

This assessment is intended to help you:

- demonstrate independent application of software engineering practices
- produce clear and readable code structure
- show organised development through version control
- demonstrate readiness for more advanced software engineering work

2. Project Requirements

A.DevelopmentApproach andVersion Control

- Git must be used for version control throughout the project.
- The repository should show meaningful commit activity reflecting incremental development.
- Development progress should be visible through commit history and task tracking or user stories.

- Version control is expectedto support organised development rather than act as a final upload mechanism.

B. Testing and Automation

- Unit tests must be implemented using a testing framework such as JUnit.
- Tests should verify core system behaviour by covering selected functionality of the system.
- Continuous integration should be used to automatically build the project and run tests, supporting reliable and repeatable verification.
- Testing in this assessment is intended to demonstrate understanding of verification and reliability rather than exhaustive test coverage or advanced testing techniques.

3. Deliverables

You will submit two deliverables.

1. Video Demonstration

A recorded video demonstrating the system and explaining your work.

The video should:

- show the system's behaviour clearly
- explain how the system is structured
- demonstrate key technical decisions
- reflect a clear and professional standard of technical communication

The video must be no longer than 10 minutes. There is no minimum length, but it must be sufficient to clearly demonstrate system behaviour and explain the implementation.

2. Zipped Source Code

You must submit a ZIP file containing:

- the complete source code of the project

- a README file

The README should include:

- the GitHub repository link
- brief instructions for running the system
- a short overview of system structure and main components

The code should be readable, clearly structured, and consistent with what is shown in the video. As no separate technical document is required, clarity and readability of the code form an important part of the submission.

4. Marking Scheme

The coursework is marked out of 100 marks, distributed as follows:

- Product Functionality and Demonstrated Behaviour – 25 marks
- Software Design and Code Quality – 25 marks
- Testing and Continuous Integration – 20 marks
- Version Control and Development Evidence – 15 marks
- Technical Communication and Demonstration Clarity – 15 marks

Detailed marking rubrics for each component are provided on the next page.

	Fail		Pass				
	0-29%	30-39%	40-49%	50-59%	60-69%	70-79%	80-100%
1. Product Functionality & Technical Readiness (25 Marks) Demonstrating a working system that implements the intended functionality described in the given scenario, showing correct and reliable behaviour across representative workflows.	The system cannot be demonstrated or is largely non-functional. Core functionality is missing or fails during execution.	Very limited functionality is shown. Major features are incomplete or unreliable.	Basic functionality is present but several core features are incomplete or inconsistent.	The system is mostly functional. Core features operate but noticeable issues or gaps remain.	The system demonstrates correct behaviour across most core scenarios with only minor issues.	A near-complete and reliable system is demonstrated, showing consistent and correct functionality across representative scenarios.	An exceptionally complete and reliable system is demonstrated. Functionality is consistently correct and reflects strong technical readiness.
2. Software Design & Code Quality (25 Marks) Demonstrating and explaining system structure, design decisions, and overall code quality, including appropriate use of architectural or design patterns and absence of significant code smells.	System structure is unclear or incorrect. Design decisions are absent or poorly applied. Code quality is poor and difficult to maintain.	Minimal evidence of structured design. Code organisation is inconsistent with several design issues.	Basic structure is present but explanation and justification of design decisions are limited. Some code quality issues or smells remain.	Clear structure with acceptable separation of responsibilities. Design decisions are explained at a basic level and code quality is generally acceptable.	Good structural design with clear organisation and maintainability. Code is readable and largely free from significant issues.	Well-structured design with clear rationale. Code organisation demonstrates strong maintainability and sound engineering practice.	Exemplary design quality. Design decisions are clearly justified and the codebase reflects professional standards with no significant code smells.
3. Testing & Continuous Integration (20	Testing is absent or non-functional. No meaningful	Limited testing with little understanding of automated	Basic tests exist but cover only minimal functionality.	Functional tests are present for some features, but	Adequate automated testing of representative	Strong testing approach with reliable automated	Comprehensive and well-structured testing approach.

Marks) Demonstrating the use of unit testing and continuous integration to automatically build the system and run tests verifying core system behaviour.	automated verification is present.	testing practices. No effective automation is demonstrated.	Automation is incomplete or unreliable.	automation is limited or inconsistent.	features is demonstrated. A functioning continuous integration setup is present that automatically builds and runs tests. Well-	execution. Continuous integration supports consistent verification of system behaviour.	Automated testing and continuous integration are effectively integrated to ensure consistent software quality and reliability.
4. Agile Practice, Version Control & Development Evidence (15 Marks) Demonstrating organised development through version control, task tracking, and clear progression of implementation.	Little or no meaningful use of version control. Development history is unclear or missing.	Minimal commit history with limited evidence of structured development. Task tracking or planning is unclear.	Basic commit history exists but shows limited progression or organisation. Task descriptions are minimal.	Reasonable use of version control showing incremental development. Task tracking or user stories are present.	maintained repository with meaningful commits reflecting iterative progress. Tasks or user stories show clear development intent.	Strong evidence of organised and disciplined development. Commit history clearly reflects incremental implementation and task progression.	Exemplary version control practice demonstrating clear, disciplined, and traceable development supported by well-maintained task tracking.
5. Technical Communication & Demonstration Clarity (15 Marks) Clearly explaining system behaviour, technical decisions, and engineering choices in a structured and understandable manner.	Explanation is unclear or disorganised. Technical reasoning is missing or incorrect.	Limited explanation of functionality or decisions. Structure is difficult to follow.	Basic explanation is provided but lacks clarity or depth. Technical reasoning is only partially explained.	Demonstration is generally clear with acceptable explanation of functionality and decisions.	Clear and structured explanation showing good understanding of system behaviour and technical choices.	Very clear and well-structured explanation demonstrating strong understanding and justification of technical decisions.	Exceptionally clear and professional technical communication demonstrating deep understanding and confident justification of engineering choices.

5. Submission Requirements

The video must be submittedusing QMplus Media (https://media.qmplus.qmul.ac.uk/). There will be a separate submission area for video submissions. Further information will be provided in the video submission area on QMplus.

All source code must be submitted as a single ZIP file. The submitted code must match the system demonstrated in the video.
The ZIP file must include the complete project repository and a README file containing:

- the GitHub repository link
- instructions for running the system
- a brief overview of the system structure and main components

The submission should be self-contained and must not rely on external resources other than the GitHub repository link. Any linked resources must be publicly accessible. Resources that cannot be accessed will not be considered during marking.

7. Late Submissions and Extenuating Circumstances

Assessmentssubmitted after the published deadlinewill bemarked as late and will receive a penalty, unless you have approved Extenuating Circumstances (EC).

You may submit up to seven calendar days after the deadline, but a penalty of 5% of the total marks available will be applied to the assessment for each 24-hour period (or part of it) that the work is late (e.g., five marks deducted each day from a total of 100).

After seven calendar days, the assessment will be recorded as a non-submission and given a mark of zero.

For more information, you can find the latest Assessment Handbook and Extenuating Circumstances policy via the QMUL policies page: https://www.qmul.ac.uk/governance-and-legal-services/policy/policies-by-category/.

8. Knowledge, Skills, and Behaviours (Apprenticeship v1.2)

Incombination,theassessmentsfor thismodulearedesignedto helpyouengagewith the following Knowledge, Skills, and Behaviours (KSBs) from your apprenticeship standard:

Knowledge

- K21 How to operate at all stages of the software development life cycle and how each stage is applied in a range of contexts. For example, requirements analysis, design, development, testing, implementation.
- K22 Principles of a range of development techniques, for each stage of the software development cycle that produce artefacts and the contexts in which they can be applied. For example UML, unit testing, programming, debugging, frameworks, architectures.
- K23 Principles of a range of development methods and approaches and the contexts in which they can be applied. For example Scrum, Extreme Programming, Waterfall, Prince2, TDD.
- K24 How to interpret and implement a design, compliant with functional, non-functional and security requirements including principles and approaches to addressing legacy software development issues from a technical and socio-technical perspective. For example architectures, languages, operating systems, hardware, business change.

- K26 How to select and applya range of software tools used in Software Engineering.
- K28 Approaches to effectiveteam work and the range of software development tools supporting effective teamwork. For example, configuration management, version control and releasemanagement.

Skills

- S17 Provide recommendations as to the most appropriate software engineering solution.
- S19 Implement software engineering projects using appropriate software engineering methods, approaches and techniques.
- S20 Respond to changing priorities and problems arising within software engineering projects by making revised recommendations, and adapting plans as necessary, to fit the scenario being investigated.
- S21 Determine, refine, adapt and use appropriate software engineering methods, approaches and techniques to evaluate software engineering project outcomes.
- S22 Evaluate learning points arising from software engineering work undertaken on a project including use of methods, analysis undertaken, selection of approach and the outcome achieved, in order to identify both lessons learnt and recommendations for improvements to future projects.

9. Final Note

This assessment rewards clarity, structure, and professional judgement. Marks are awarded for demonstrating sound software engineering practices and a clear understanding of the design and implementation decisions made throughout the development process.